UNIVERSIDAD MESOAMERICANA

Técnico en Desarollo de Software

CURSO: Desarrollo web II	**SEMESTRE:** 02-2026	**DURACIÓN:** 100 minutos	**FECHA:** 07-11-2026	**EVALUACIÓN:** Examen final

Nombre del estudiante: Diego Alejandro Bran Valdéz

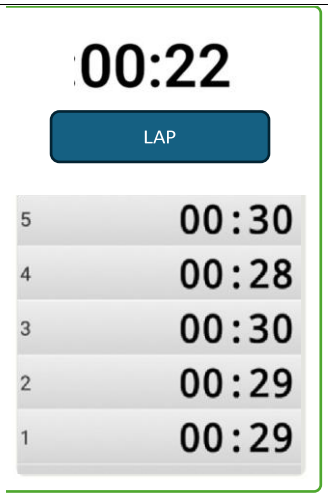
SERIE I: (Pregunta directa) Lea cuidadosamente las preguntas y responda como corresponde.

1. ¿Explique todos los patrones de estado existentes en REACT y para qué sirve cada uno de ellos?
2. ¿Cuáles son las diferencias entre los hooks (useState y useReducer)?
3. ¿Cuál es el propósito primordial de reactRouter, de un ejemplo de ello?
4. Liste el scaFolding recomendado para un proyecto de React escalable en caso de ser una aplicación con una gran cantidad de componentes. (Describa cada parte del scafold)
5. Describa como complementan los conceptos de (promise, fetch y axios).

SERIE II: Desarrolle lo solicitado

Codifique un software de REACT con el siguiente comportamiento:

- Que tenga un Timestop que maneje, segundos y minutos tal cual se ve en la imagen del lado derecho.
- El cronometro debe iniciar su funcionamiento al cargar la pantalla.
- Los ":" entre minutos y segundos deben parpadear cada segundo transcurrido, es decir un segundo deben ser visibles y al siguiente segundo no.
- Cuando se presione el botón "LAP", se debe almacenar el tiempo transcurrido en una lista y mostrarse bajo el botón mencionado.
- Cuando se presione el botón "LAP" el cronometro debe regresar al valor 00:00.



UNIVERSIDAD MESOAMERICANA

Técnico en Desarollo de Software

Liste cada uno de los errores del siguiente código deREACT proporcionado y cuál sería la solución a cada uno de los errores identificados (Identifique como mínimo 4 errores):

```
import { useState, useEffect } from "react";

function ExampleApp() {
  const [count, setCount] = useState();
  const [name, setName] = useState("Student");

  useEffect(() => {
    console.log("Component mounted");
    if (count > 3) {
      setName("Done");
    }
    setCount(count + 1);
  });

  useEffect(() => {
    const timer = setInterval(() => {
      console.log("Running...");
    }, 1000);
  }, [name]);

  return (
    <div>
      <h2>Hello {name}</h2>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}

export default ExampleApp;
```

- **SERIE I:**

1.

- **useState/ useReducer:** Maneja datos dentro de un solo componente
- **Lifted State**: Eleva el estado al componente padre común para sincronizarlo entre componentes hermanos mediante props.
- **Global State**: Comparte datos en toda la aplicación sin pasar props manualmente (ej. usuario autenticado, carrito, modo oscuro).
- **Server State**: Gestiona datos remotos de una API con caché, sincronización asíncrona y revalidación automática.
- **URL State**: Almacena el estado en la URL para compartir o navegar filtros, paginación o pestañas.
- **Derived State:** Valores calculados directamente en el render a partir de props o estados existentes.

2.

- **Complejidad:** useState se usa para estados simples e independientes (primitivos); useReducer para estados complejos con múltiples subvalores o dependencias.
- **Actualización:** useState actualiza el valor directamente con una función (setState(valor)); useReducer envía acciones con dispatch(action) que procesa una función reductora pura.
- **Separación de lógica:** useReducer permite extraer y testear la lógica de transición fuera del componente; en useState la lógica vive dentro del componente.
- **Optimización:** useReducer ofrece un dispatch con referencia estable garantizada, ideal para pasarlo por props o context a componentes hijos sin causar re-renders.

3. Permitir la navegación del lado del cliente en aplicaciones SPA, cambiando vistas y URLs sin recargar la página en el navegador.

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">HomePage</Link>
        <Link to="/about">AcercaDe</Link>
      </nav>
      <Routes>
        <Route path="/" element={<homePage />} />
        <Route path="/about" element={acercaDet />} />
      </Routes>
    </BrowserRouter>
  );
}
```

4. Features Arq.

- **src/features/:** Módulos divididos por dominio de negocio (ej. auth, products); cada uno contiene sus propios componentes, hooks y servicios locales.
- **src/components/:** Componentes UI globales, atómicos y reutilizables en toda la app (botones, inputs, modales).
- **src/hooks/:** Custom hooks de uso global o compartido (ej. useDebounce, useLocalStorage).
- **src/services/:** Configuración de clientes HTTP (instancias base de Axios o Fetch) y llamadas a endpoints.
- **src/context/:** Proveedores de estado global mediante Context API (ej. AuthContext, ThemeContext).
- **src/routes/:** Configuración central de rutas y protección de accesos/guards.
- **src/assets/:** Recursos estáticos globales (imágenes, SVGs, fuentes).
- **src/utils/:** Funciones de utilidad pura y formateo (fechas, validaciones regex).

5. La Promise es el objeto nativo asíncrono sobre el cual se construyen tanto Fetch como Axios; Fetch es la interfaz nativa del navegador para peticiones HTTP usando promesas, mientras que Axios es una biblioteca externa que amplía y resuelve las carencias de Fetch al incorporar sobre esas promesas el parseo automático de JSON, interceptores para tokens y el rechazo automático ante errores HTTP.

SERIE II:

LINK DE REPOSITORIO: https://github.com/elPatronDB/2026-2-desarrollo-web-2-asincrono/tree/91a3d99c960c06d38c492dcf2a5220a1e9264396/NOVIEMBRE/examen-final

SERIE III:

1. Llave de cierre extra tras el primer hook: Deja el segundo `useEffect` y el `return` fuera de la función.

Solución: Eliminar esa llave } para que todo quede dentro de `ExampleApp`.

2. Hook fuera del componente: El segundo `useEffect` queda huérfano y rompe las reglas de React.

Solución: Meterlo dentro del cuerpo del componente.

3. Loop infinito: El primer `useEffect` no tiene arreglo de dependencias (`[]`) y actualiza estado (`setCount`).

Solución: Añadirle dependencias `[count]` o `[]`.

4. Fuga de memoria: El `setInterval` no se destruye al desmontar o cambiar `name`.

Solución: Retornar `() => clearInterval(timer)`.

5. Sintaxis en asignación: Dice `const timer setInterval` sin el signo `=`.

Solución: Escribir `const timer = setInterval(...)`.

6. Sintaxis rota en JSX: `return ();` está vacío y el botón tiene caracteres extra (`onClick $=\{...`).

Solución: Envolver el JSX en el `return (...)` y corregir a `onClick={() => setCount(count + 1)}`.

7. Estado no inicializado: `useState()` inicia en `undefined` y al sumar da `NaN`.

Solución: Inicializarlo con cero: `useState(0)`.